

A GROUND-UP DEEP DIVE

The Comprehensive LLM Study Guide

From "what is a word made of numbers?" to transformers, training, and the frontier — written for a complete beginner, with every piece of math explained from zero.

Companion volume to **The Comprehensive RAG Study Guide**

Concepts • Mathematics • Intuition • Worked examples

No prior knowledge assumed

Table of Contents

Part 1 — Foundations: What a Language Model Actually Is

1.1 The one-sentence definition • 1.2 Predicting the next token • 1.3 Probability, and turning scores into odds • 1.4 Why this simple game produces intelligence • 1.5 Base model vs. assistant

Part 2 — The Road Here: A Short History

2.1 Counting words (n-grams) • 2.2 Meaning as geometry (Word2Vec) • 2.3 Reading in order (RNN/LSTM) • 2.4 The wall • 2.5 Attention (2014) • 2.6 "Attention Is All You Need" (2017) • 2.7 BERT vs. GPT • 2.8 The scaling era • 2.9 RLHF & ChatGPT

Part 3 — Tokenization: Turning Text Into Numbers

3.1 Why not words or letters • 3.2 Subwords & Byte-Pair Encoding • 3.3 The vocabulary trade-off • 3.4 Special tokens • 3.5 Why tokens are money

Part 4 — Embeddings & Position

4.1 From token IDs to vectors • 4.2 What lives in an embedding • 4.3 The residual stream • 4.4 The position problem • 4.5 Positional encodings (sinusoidal, learned, RoPE, ALiBi)

Part 5 — The Transformer, From Zero

5.1 The big picture • 5.2 Self-attention • 5.3 Q, K, V • 5.4 Softmax, in depth • 5.5 Scaled dot-product • 5.6 Multi-head • 5.7 Causal masking • 5.8 The feed-forward network, in depth • 5.9 Residuals & LayerNorm • 5.10 The full block • 5.11 Encoder vs. decoder

Part 6 — Attention at Scale

6.1 The n^2 problem • 6.2 The KV cache • 6.3 MQA & GQA • 6.4 FlashAttention • 6.5 Sparse & sliding-window • 6.6 Long context

Part 7 — Anatomy of an LLM: Reading the Spec Sheet

7.1 What "7B parameters" counts • 7.2 Where the parameters live • 7.3 Depth, width, heads • 7.4 Context, vocab, tied weights • 7.5 FLOPs • 7.6 Mixture-of-Experts • 7.7 Reading a real spec sheet

Part 8 — Pretraining: How a Model Learns

8.1 The objective • 8.2 Loss & cross-entropy • 8.3 Gradient descent • 8.4 Backpropagation, in depth • 8.5 The data pipeline • 8.6 Distributed training • 8.7 Scaling laws • 8.8 Emergence

Part 9 — From Base Model to Assistant

9.1 Why a base model isn't an assistant • 9.2 SFT • 9.3 RLHF • 9.4 DPO • 9.5 Constitutional AI • 9.6 What alignment does & doesn't fix

Part 10 — Inference & Decoding

10.1 The autoregressive loop • 10.2 Greedy & beam • 10.3 Temperature • 10.4 Top-k, top-p, min-p • 10.5 Repetition control • 10.6 Prefill vs. decode • 10.7 Speculative decoding

Part 11 — Making Models Smaller & Faster

11.1 Quantization • 11.2 Distillation • 11.3 Pruning • 11.4 LoRA & QLoRA

Part 12 — Using LLMs Well

12.1 In-context learning • 12.2 Chain-of-thought • 12.3 System prompts • 12.4 Tools & structured output • 12.5 The bridge to RAG

Part 13 — Capabilities, Limits & Failure Modes

13.1 Why hallucination is structural • 13.2 Knowledge cutoff • 13.3 Lost in the middle • 13.4 Arithmetic & reasoning • 13.5 Bias & safety • 13.6 Benchmarks

Part 14 — The Frontier

14.1 Reasoning models • 14.2 Multimodality • 14.3 Agents • 14.4 Frontier-scale MoE • 14.5 Open vs. closed

Part 15 — A Beginner's Checklist & Glossary

Foundations: What a Language Model Actually Is

Before transformers, before training, before any math, we need one idea. Everything in this guide is built on it, so we will go slowly.

1.1 The one-sentence definition

A **language model** is a machine that, given some text, predicts what text is likely to come next. That is the whole job. A model that has read most of the internet has become extraordinarily good at this one game — and it turns out that being good at "guess the next bit of text" forces a machine to learn grammar, facts, reasoning patterns, translation, and much more, because all of those things are needed to guess well.

INTUITION

Play this in your head: "*The capital of France is ____.*" You said "Paris" instantly. You just *were* a language model. You used everything you know — geography, grammar, the shape of the sentence — to predict the next word. An LLM does the same thing, only across billions of such guesses per second, having practised on trillions of words.

1.2 Predicting the next token

We said "next word," but real models work with **tokens** — chunks of text that are often whole words but sometimes pieces of words (we build these carefully in Part 3). For now, think "token $\approx$ word." The model reads a sequence of tokens and outputs a guess for the next one. Then that guess is added to the sequence, and the model guesses again. This repeat-forever loop is how a model writes a whole paragraph from a short prompt — one token at a time. This is called being **autoregressive**: each new output feeds back in as input.

```
"The cat sat on the" → [ LANGUAGE MODEL ] → guess: "mat" (most likely)
                                     "floor", "sofa" (also possible)
"The cat sat on the mat" → [ MODEL ] → guess: "."
... and so on, token by token, until the text is complete.
```

1.3 Probability — turning scores into odds

The model does not output a single word. It outputs a **probability** for *every* token in its vocabulary at once. If the vocabulary has 50,000 tokens, the model produces 50,000 numbers that sum to 1 (i.e. to 100%). Maybe "mat" gets 0.31, "floor" gets 0.12, "banana" gets 0.0000004, and so on. This full list is called a **probability distribution** over the next token.

THE CORE EQUATION

We write the model's job as: estimate the probability of the next token x_t given all the tokens before it.

$$P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

Read the vertical bar "|" as "given." So this reads: "the probability of token number t , given every token that came before it." The probability of a whole sentence is just these one-step probabilities multiplied together, walking left to right.

Internally the model first produces raw scores — one score per possible token — called **logits**. Logits can be any number (big, small, negative). A function called **softmax** then squashes those raw scores into clean probabilities that are all positive and sum to 1. Softmax is so central that we devote a full, from-zero section to it in Part 5.4. For now, remember the pipeline:

text → model → logits (raw scores, any value) → SOFTMAX → probabilities (sum to 1) → pi

1.4 Why this simple game produces intelligence

It is fair to be sceptical. How does "predict the next token" become something that can write code, explain physics, or draft an email? The answer is that **prediction is a demanding master**. To reliably predict the next token in a physics textbook, the model must internalise physics. To finish a line of code, it must learn syntax and logic. To continue a story sensibly, it must track characters and cause and effect. The single objective of good prediction quietly forces the model to build compressed internal representations of how the world and language work. Nobody programmed those skills in; they emerged as a side effect of getting very, very good at guessing.

EXAMPLE

To finish "*The doctor picked up her __,*" the model must know that "doctor" can be female (it should not fight the word "her"), that a doctor plausibly picks up a *stethoscope* or *chart*, and that the sentence wants a noun next. Doing this well, billions of times over, is indistinguishable from understanding language.

1.5 Base model vs. assistant

A raw model trained only to predict text is called a **base model** (or "foundation model"). It is a brilliant autocomplete but an awkward conversationalist — ask it a question and it might continue with *more questions*, because that is a plausible continuation of text on the internet. A second stage of training (Part 9) teaches it to *follow instructions* and behave like a helpful **assistant**. ChatGPT, Claude, and Gemini are assistants built on top of base models. Keep this two-stage picture in mind: **pretraining** creates raw capability; **post-training** shapes it into something you can talk to.

KEY TAKEAWAYS

- A language model predicts the next token given the previous ones — that is the entire task.
- It outputs a full probability distribution over the vocabulary, not a single word.
- Raw scores (logits) become probabilities via **softmax**.
- Generating text is a loop: predict, append, repeat (**autoregressive**).
- Being excellent at prediction forces the model to learn grammar, facts, and reasoning.
- A **base model** autocompletes; a second training stage turns it into an **assistant**.

The Road Here: A Short History

Transformers did not appear from nowhere. Each earlier idea solved one problem and exposed the next. Knowing this chain makes the transformer feel inevitable rather than magical.

2.1 Counting words: n-gram models (1950s-2000s)

The earliest language models simply **counted**. To guess the word after "the cat sat on the," an **n-gram** model looks at how often each word followed that phrase in its text collection and picks the most common. An n-gram is just a window of n consecutive words. This works for short, common phrases and fails everywhere else: it has no notion of meaning (it cannot see that "sofa" and "couch" are similar), and it drowns in the sheer number of possible word combinations — most longer phrases were never seen even once, so the model has nothing to count. This is the **sparsity** problem.

2.2 Meaning as geometry: word embeddings (2013)

The breakthrough of **Word2Vec** (2013) was to represent each word as a **vector** — a list of numbers, a point in space — learned so that words used in similar contexts land near each other. Suddenly "king" and "queen" sit close together, and meaning became something you could measure with geometry. The famous result was that vector arithmetic captured analogies:

$$\text{vector}(\text{"king"}) - \text{vector}(\text{"man"}) + \text{vector}(\text{"woman"}) \approx \text{vector}(\text{"queen"})$$

This idea — **meaning is a location in space** — never went away. It is the foundation of the embeddings inside every modern LLM (Part 4) and of retrieval systems (see the RAG guide). But a plain Word2Vec vector is *static*: the word "bank" gets one vector whether you mean a riverbank or a savings bank. Context was still missing.

2.3 Reading in order: RNNs and LSTMs (1990s-2016)

To use context, a model must read a sentence *in order* and carry a memory of what it has seen. The **Recurrent Neural Network (RNN)** did exactly this: it processes one word, updates a hidden "memory" vector, processes the next word, updates memory again, and so on. The **LSTM** (Long Short-Term Memory) added gates that let the network choose what to remember and what to forget, which helped it hold information over longer spans.

2.4 The wall RNNs hit

RNNs had two crippling problems that motivated everything after them:

- **They are sequential.** Word 100 cannot be processed until words 1–99 are done. You cannot spread the work across a modern GPU's thousands of parallel cores, so training on huge text is painfully slow.
- **They forget.** Squeezing an entire sentence's history into one fixed-size memory vector means early words fade by the time you reach the end — the "long-range dependency" problem. Connecting a pronoun to a noun mentioned 40 words earlier was unreliable.

2.5 Attention arrives (2014)

The first fix was **attention**. Instead of cramming everything into one memory vector, let the model, at each step, *look back* at all previous words and decide which ones are relevant right now — paying more "attention" to some than others. A translation model producing the French word for "bank" could look straight back at "river" earlier in the sentence. Attention was originally a helper bolted onto RNNs. Then came the radical question: what if attention is not a helper, but the whole engine?

2.6 "Attention Is All You Need" (2017)

In 2017 a Google team published a paper with that title, introducing the **Transformer**. Its bet: throw away recurrence entirely and build the model out of attention alone. Because attention looks at all words simultaneously rather than one after another, the whole sequence can be processed **in parallel** — a perfect match for GPUs. This single change unlocked training on internet-scale data, and every major LLM since (GPT, BERT, Claude, Gemini, Llama) is a transformer. Part 5 builds this architecture from the ground up.

WHY IT MATTERED

The transformer's superpower is not just accuracy — it is **parallelism**. RNNs forced computation into a slow single-file line. Transformers let you use a whole GPU at once, which is what made training on trillions of words financially and physically possible. Scale caused the revolution; the transformer is what made scale possible.

2.7 Two families: BERT vs. GPT (2018)

The transformer split into two lineages, and the difference echoes through the whole field:

	BERT (encoder)	GPT (decoder)
Reads	Whole sentence at once, both directions	Left to right only
Trained to	Fill in blanked-out words	Predict the next token
Best at	Understanding / classifying text	Generating text
Descendants	Search, embeddings, classifiers	Every chat LLM

Because we care about models that *write*, this guide follows the **GPT / decoder** line. BERT-style encoders still power the embedding models behind retrieval, so they are far from obsolete — they simply do a different job.

2.8 The scaling era (2019-2022)

Once the transformer made scale possible, researchers asked: what happens if we just make everything bigger — more parameters, more data, more compute? The answer, documented in **scaling laws** (Part 8.7), was startling: performance kept improving smoothly and predictably. GPT-2 (2019, 1.5 billion parameters) could write coherent paragraphs. GPT-3 (2020, 175 billion) could, without any extra training, do tasks it was merely *shown* a few examples of in the prompt — a skill called **in-context learning** (Part 12.1). Nobody built that skill in; it emerged from scale.

2.9 Making it helpful: RLHF and ChatGPT (2022)

GPT-3 was powerful but hard to steer. The final ingredient was teaching models to *follow instructions* and match human preferences, using a technique called **RLHF** — Reinforcement Learning from Human Feedback (Part 9.3). Humans ranked model answers; the model learned to produce the kind humans preferred. The result, released as **ChatGPT** in late 2022, was the same underlying technology as GPT-3 but wrapped into something that felt like talking to a helpful person. That usability — not a new architecture — is what put LLMs in front of hundreds of millions of people.

KEY TAKEAWAYS

- **n-grams** counted words — no meaning, hopeless sparsity.
- **Word2Vec** made meaning geometric: words became points in space.
- **RNNs/LSTMs** read in order but were slow (sequential) and forgetful (long-range).
- **Attention** let a model look back at any earlier word directly.
- The **2017 Transformer** made attention the whole engine, enabling GPU-scale parallel training.
- **BERT** (understand) and **GPT** (generate) split off; chat LLMs are the GPT line.
- **Scaling** bigger models on more data kept improving them; **RLHF** made them helpful, giving us ChatGPT.

Tokenization: Turning Text Into Numbers

A neural network only does arithmetic. It cannot see letters. So the very first step in any LLM is converting text into numbers — and how we chop text up has surprising consequences.

3.1 Why not just words, or just letters?

Two obvious ideas both fail:

- **One number per word.** The vocabulary becomes enormous and open-ended (new slang, typos, names, other languages appear constantly). And a word the model never saw in training — an "out-of-vocabulary" word — becomes an unrepresentable blank.
- **One number per letter.** The vocabulary is tiny and nothing is ever unknown, but now the model must reconstruct every word from scratch and sequences become extremely long, which is slow and makes learning harder because meaning is spread thinly across many tiny pieces.

The winning compromise sits in between: **subword tokenization**.

3.2 Subwords and Byte-Pair Encoding (BPE)

The idea: keep common words whole ("the," "dog," "running") but break rare words into meaningful pieces ("tokenization" → "token" + "ization"; "unhappiness" → "un" + "happiness"). This gives a manageable vocabulary (typically 30,000–150,000 tokens) while *never* being stuck — worst case, a bizarre string falls back to individual characters or bytes.

The most common method, **Byte-Pair Encoding (BPE)**, learns these pieces automatically. It is beautifully simple:

1. Start with every individual character as a token.
2. Scan the training text and find the *most frequent adjacent pair* of tokens.
3. Merge that pair into a single new token, and add it to the vocabulary.
4. Repeat thousands of times, each merge creating a slightly bigger piece.

WORKED EXAMPLE

Suppose your text is full of the word "low," "lower," "lowest." BPE starts with letters: `l o w e r`. It notices `l + o` is very common → merges to `lo`. Then `lo + w` → `low`. Now "low" is one token, and "lower" is just `low + er`. The common stem got its own token; the endings are reused across many words. Frequency, not a dictionary, decided the vocabulary.

Related schemes you will hear about: **WordPiece** (used by BERT, merges by likelihood rather than raw frequency), **Unigram** (starts big and prunes), and **SentencePiece** (a toolkit that treats the space character as just another symbol, so it works cleanly for languages that don't use spaces). Modern GPT-style models use **byte-level BPE**, which operates on raw bytes so it can encode *literally any* input — every emoji, script, and control character — with zero unknowns.

3.3 The vocabulary-size trade-off

Smaller vocabulary (e.g. 32k)	Larger vocabulary (e.g. 128k)
Fewer parameters in the embedding table	More parameters, more memory
Longer token sequences (words split more)	Shorter sequences — more text per token
Cheaper per token, but more tokens needed	Fewer tokens for the same text → faster, cheaper context

Bigger vocabularies have trended in over time because fitting more text into fewer tokens directly saves money and lets more content fit in the context window. There is no universally "correct" size; it is an engineering trade-off tuned per model.

3.4 Special tokens

Beyond ordinary text, the vocabulary includes **special control tokens** that carry structure: markers for the start/end of a document, an end-of-sequence token that tells the model to stop generating, and — crucially for chat models — role markers that delimit the system prompt, the user's turn, and the assistant's turn. When you "talk to" a chat model, your message is quietly wrapped in these special tokens so the model knows who is speaking.

3.5 Why tokens are money (and other practical facts)

- **Billing and limits are in tokens, not words.** As a rough rule for English, 1 token $\approx$ 0.75 words, or $\sim$ 4 characters. "Hello" is one token; "antidisestablishmentarianism" is several.
- **The context window is measured in tokens.** A "128k context" means 128,000 tokens of prompt plus response, not 128,000 words.
- **Tokenization explains famous failures.** When a model struggles to count the letters in "strawberry," it is partly because it never sees the letters — it sees the tokens `str`, `aw`, `berry`. The individual r's are hidden inside chunks.
- **Non-English text often costs more.** Languages under-represented in the tokenizer's training get split into more tokens, so the same sentence uses more tokens and more money.

KEY TAKEAWAYS

- Networks do arithmetic, so text must become numbers; tokenization does this first.
- **Subword** tokens balance a manageable vocabulary against never being stuck on unknown words.
- **BPE** learns tokens by repeatedly merging the most frequent adjacent pair.
- Vocabulary size is a trade-off between model size and sequence length.
- Special tokens encode structure like stop-signals and chat roles.
- Tokens are the unit of cost, context limits, and several well-known model quirks.

Embeddings & Position

Tokenization gave each token an ID number like 4021. But an ID is just a name — it carries no meaning. Embeddings turn each ID into a rich vector of meaning, and positional encodings tell the model where each token sits.

4.1 From token IDs to vectors

The model keeps a giant lookup table called the **embedding matrix**: one row per vocabulary token, each row a vector of, say, 4,096 numbers. Tokenization converts "cat" to an ID; the model looks up that ID's row and retrieves its **embedding vector**. That vector — not the ID — is what flows into the network. The numbers in these vectors are *learned during training*; they start random and are gradually adjusted so that useful meaning ends up encoded in them.

THE SHAPES INVOLVED

Two numbers define the table. The **vocabulary size** V (how many tokens exist, e.g. 50,000) and the **model dimension** d (the length of each vector, e.g. 4,096, often written d_{model}). The embedding matrix is therefore a grid of $V \times d$ numbers. A sequence of n tokens becomes a grid of $n \times d$ numbers: n vectors, one per token. Everything the transformer does is transform this grid.

4.2 What actually lives in an embedding

Because vectors are points in space, direction and distance encode meaning. Tokens with similar meaning end up with similar vectors (small angle between them). Different directions come to represent different properties — loosely, one direction might track "royalty," another "plural," another "past tense." You never label these directions; training discovers them because they help prediction. This is the same "meaning as geometry" idea from Word2Vec (Part 2.2), now learned jointly with the whole model and, unlike Word2Vec, refined by context as the vector passes through the network's layers.

4.3 The residual stream: a mental model

Here is a picture that will pay off repeatedly. Imagine each token's vector as a **running notepad** that travels up through the network's layers. Every layer *reads* the notepad, does some thinking, and *adds* its conclusions back onto the same notepad rather than replacing it. This shared, added-to vector is called the **residual stream**. It starts as "this token is 'cat' at position 3" and, layer by layer, accumulates richer notes: "'cat', subject of the sentence, associated with 'sat', the thing sitting on the mat." By the top, the notepad holds everything needed to predict what comes next. We will see exactly how layers add to it in Part 5.9.

4.4 The position problem

Here is a subtlety that trips everyone up. The transformer's attention mechanism (Part 5) looks at all tokens *simultaneously* and, by itself, has **no sense of order** — to raw attention, "dog bites man" and "man bites dog" contain the same set of tokens and look identical. That

is a disaster for language, where order is meaning. Because we gave up the sequential reading of RNNs to gain parallelism, we must *put order back in by hand*. That is what positional encoding does.

4.5 Positional encodings

The fix is to add a second vector to each token's embedding that depends on its *position*, so the token at slot 1 and the same token at slot 5 enter the network as different vectors. Several schemes exist:

- **Sinusoidal (original transformer).** Use a fixed pattern of sine and cosine waves of different frequencies to generate a unique vector for each position. Clever because the relationship between positions is smooth and the model can, in principle, extend to lengths it never saw.
- **Learned absolute positions.** Simply keep a second lookup table — one learned vector per position — and add it in. Simple and effective, but it cannot handle positions beyond the maximum length seen in training.
- **RoPE (Rotary Position Embedding).** The modern default in most open LLMs. Instead of *adding* a position vector, it *rotates* each token's query and key vectors by an angle proportional to its position. The elegant consequence is that when two tokens interact, only their *relative* distance affects the result — exactly what language cares about. RoPE also extends to longer contexts more gracefully.
- **ALiBi.** Skips position vectors entirely and instead adds a small penalty to attention that grows with distance, gently biasing each token to favour nearby tokens. Simple and extrapolates well.

WHY YOU KEEP HEARING "ROPE"

When a model advertises a longer context window (say, from 8k to 128k tokens), the trick is very often adjusting RoPE so its rotations still make sense at the new, larger distances. Position encoding is quietly one of the most active areas of long-context engineering.

KEY TAKEAWAYS

- An **embedding** turns each token ID into a learned vector of meaning via a lookup table.
- The table is $V \times d$; a sequence becomes an $n \times d$ grid.
- Similar meanings $\rightarrow$ similar vectors; directions encode properties, all learned.
- The **residual stream** is a per-token notepad that every layer reads from and adds to.
- Attention is order-blind, so **positional encoding** injects each token's location.
- **RoPE** (rotate by position, encoding relative distance) is the modern default.

The Transformer, From Zero

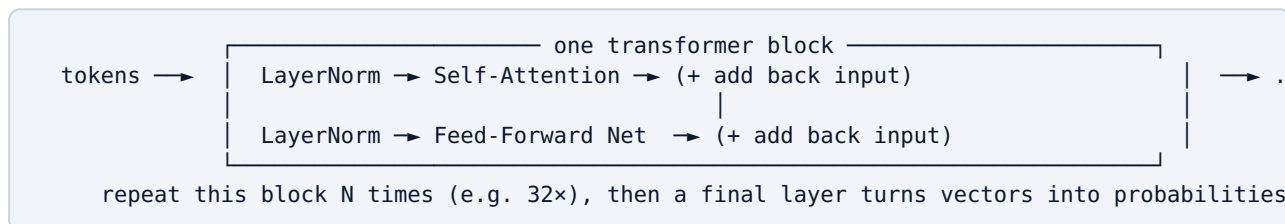
This is the engine. We will build it one piece at a time, and we will not skip the math — but every symbol will be explained the first time it appears. Take this part slowly; everything after it is easier.

5.1 The big picture

A transformer is a tall stack of identical **layers** (also called **blocks**). The token embeddings from Part 4 enter at the bottom; they flow up through every layer; at the top, the final vectors are turned into next-token probabilities. Each layer contains exactly two working parts, applied in order:

1. **Self-attention** — lets each token gather information from other tokens. This is how words share context ("her" finds "doctor").
2. **A feed-forward network (FFN)** — lets each token, on its own, do further processing on what it just gathered. This is where much of the model's "knowledge" is stored.

Around each of those two parts sit two supporting mechanisms — a **residual connection** and a **normalization** step (Part 5.9) — that keep training stable. That is the entire recipe, repeated dozens of times.



5.2 Self-attention: the core intuition

Attention answers one question for every token: *"Which other tokens should I pay attention to in order to understand myself better, and how much?"* Consider "The animal didn't cross the street because **it** was too tired." What does "it" refer to? A human knows "the animal." Self-attention lets the vector for "it" reach back, notice that "animal" is highly relevant, and pull in information from it — so that "it" is now enriched with "animal-ness." Every token does this for every other token, in parallel.

5.3 Queries, Keys, and Values

To make "who is relevant to whom" computable, each token produces three different vectors, by multiplying its embedding by three learned matrices (W^Q , W^K , W^V). The dating-app analogy is the clearest:

- **Query (Q)** — what *this* token is looking for. ("I'm a pronoun; I'm looking for the noun I refer to.")
- **Key (K)** — what *each* token offers, as an advertisement. ("I'm a noun, an animal.")
- **Value (V)** — the actual information a token will hand over if it is attended to.

The mechanism: a token's Query is compared against every token's Key. Good matches (Query and Key pointing similar directions) get high scores. Those scores become weights, and the token collects a weighted blend of everyone's Values. A token that matched strongly contributes most of its Value.

MEASURING A MATCH: THE DOT PRODUCT

How do we score how well a Query matches a Key? With the **dot product**. For two vectors, you multiply them element by element and add up the results:

$$q \cdot k = q_1k_1 + q_2k_2 + \dots + q_dk_d$$

If $q = [2, 1, 0]$ and $k = [3, 1, 4]$, then $q \cdot k = (2 \times 3) + (1 \times 1) + (0 \times 4) = 7$. The dot product is large and positive when two vectors point the same way, near zero when they are unrelated (perpendicular), and negative when they point oppositely. So it is a natural **similarity score** — exactly what "how relevant is this Key to my Query" needs.

5.4 Softmax, in depth

The attention scores from the dot products are raw numbers — some big, some small, some negative. We need to turn them into **weights**: all positive, and summing to 1, so they behave like "what fraction of my attention goes to each token." The function that does this is **softmax**, and because it appears both here and at the model's output, we will understand it completely.

THE SOFTMAX FORMULA

Given a list of raw scores $z_1, z_2, \dots, z_n$, softmax turns score number i into:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{e^{z_1} + e^{z_2} + \dots + e^{z_n}}$$

In words: **raise e to the power of each score, then divide each result by the total**. The symbol $e \approx 2.718$ is Euler's number; e^z is the **exponential function**, which we lean on next.

Why this exact recipe? It does three jobs at once, and each is deliberate:

- **Everything becomes positive.** The exponential e^z is always positive, even when z is negative (e.g. $e^{-2} \approx 0.135$). So no matter the raw scores, every weight comes out positive.
- **Everything sums to 1.** Dividing each term by the sum of all terms guarantees the results add up to exactly 1 — a valid probability distribution.
- **Bigger scores win, gently but decisively.** Because the exponential grows fast, a score that is a bit larger than the others gets a *much* larger share — but not *all* of it. It is a "soft" maximum: it highlights the top choice without completely erasing the rest. Hence "soft-max."

SOFTMAX, WORKED BY HAND

Take three raw scores: $z = [2, 1, 0]$.

Step 1 — exponentiate each. $e^2 = 7.39$, $e^1 = 2.72$, $e^0 = 1.00$.

Step 2 — add them up. $7.39 + 2.72 + 1.00 = 11.11$.

Step 3 — divide each by the total.

$7.39 / 11.11 = \mathbf{0.665}$ • $2.72 / 11.11 = \mathbf{0.245}$ • $1.00 / 11.11 = \mathbf{0.090}$.

The results $[0.665, 0.245, 0.090]$ are all positive and sum to 1.000. Notice the input gap of just "2 vs 1" became an output share of "0.665 vs 0.245" — the largest score grabbed the lion's share. That amplification is softmax's signature.

THE TEMPERATURE KNOB (PREVIEW OF PART 10)

If you divide all scores by a number T before softmax, you control how sharp the result is. Small T makes the winner even more dominant (confident, repetitive); large T flattens the distribution (varied, creative, riskier). This one knob is the **temperature** setting you meet when generating text. Same math, used twice: inside attention, and at the output.

5.5 Putting it together: scaled dot-product attention

We now have all the pieces. Here is the complete attention operation, the single most important equation in the field:

THE ATTENTION EQUATION

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Let us read it strictly left to right, because every symbol now has a meaning we built ourselves:

- QK^T — multiply the Queries by the (transposed) Keys. This computes the dot product of *every* Query with *every* Key at once, giving a grid of raw similarity scores: row i , column j is "how much does token i care about token j ."
- $\div \sqrt{d_k}$ — divide by the square root of the key dimension. Why? When vectors are long, dot products get large just from adding up many terms, which pushes softmax into an extreme, near-all-or-nothing regime where learning stalls. Dividing by $\sqrt{d_k}$ rescales the scores back to a sane range. This is the "scaled" in scaled dot-product attention.
- **softmax(...)** — turn each row of scores into clean weights summing to 1, exactly as in 5.4.
- $\times V$ — use those weights to take a weighted average of the Value vectors. The result for each token is a blend of everyone's information, dominated by whoever it attended to most.

The output is a new set of vectors, one per token, each now enriched with context from the tokens it found relevant. That is self-attention.

5.6 Multi-head attention

One attention operation can only focus on one kind of relationship at a time. But language has many relationships running at once: grammatical agreement, who-did-what-to-whom, coreference (pronouns), topic. The fix is **multi-head attention**: run several attention operations in parallel — each called a **head**, each with its own W^Q , W^K , W^V matrices — so each head can specialise. One head might learn to track the previous adjective, another to link verbs to subjects. Their outputs are concatenated and mixed by one more matrix. If the model dimension is 4,096 and there are 32 heads, each head works in a smaller 128-dimensional slice, so the total cost is about the same as one big attention — you get specialisation for free.

5.7 Causal masking: no peeking ahead

A model that generates text left to right must never see the future while predicting it — when guessing token 5, it may use tokens 1–4 but not tokens 6 onward, which do not exist yet. We enforce this with a **causal mask**: before softmax, we set the attention scores for all "future" positions to negative infinity. After exponentiating, $e^{-\infty} = 0$, so those positions get exactly zero weight. Each token can attend only to itself and the past. (This is the one place the GPT/decoder line differs from BERT, which is allowed to look both ways because it is not generating.)

Causal mask (✓ = allowed to attend, · = blocked/future):

	The	cat	sat	on
The	[✓	·	·	·]
cat	[✓	✓	·	·]
sat	[✓	✓	✓	·]
on	[✓	✓	✓	✓]

each row sees only itself and earlier tokens

5.8 The feed-forward network, in depth

After attention has let a token gather context, the **feed-forward network (FFN)** — also called the MLP, for multi-layer perceptron — processes each token's vector *independently*. Attention mixes information *across* tokens; the FFN thinks hard about *each* token on its own. This is where a large fraction of the model's parameters live, and where much of its factual knowledge is believed to be stored.

WHAT AN FFN ACTUALLY COMPUTES

The FFN is two matrix multiplications with a non-linear function in between:

$$FFN(x) = W_2 \cdot g(W_1 \cdot x + b_1) + b_2$$

Reading it: take the token's vector x , multiply by a matrix W_1 and add a bias b_1 (this is a **linear layer**); apply a non-linear function g ; then multiply by a second matrix W_2 and add b_2 .

Two things deserve a careful walk-through: **what a matrix multiply does**, and **why the non-linear function in the middle is non-negotiable**.

A matrix multiply is just many weighted sums

A **matrix** is a grid of learned numbers (weights). Multiplying a vector by a matrix produces a new vector, where each output number is a weighted sum of all the input numbers — the weights being one row of the matrix. So "multiply by W_1 " means "compute many different weighted combinations of the token's features." Crucially, the FFN first **expands** the vector to a much larger width (typically 4× wider — e.g. $4,096 \rightarrow 16,384$), giving the network room to compute many intermediate features, then the second matrix **projects** it back down to the original width so it fits back onto the residual stream. Expand, think, compress.

Why the non-linearity is the whole point

Here is a fact that surprises everyone: if you stack two linear layers with nothing in between, the result is mathematically equivalent to a *single* linear layer. Two matrix multiplies collapse into one. So a deep stack of purely linear layers is no more powerful than one layer — it could only ever draw straight-line relationships, and language is anything but straight-line. The **non-linear activation function** g in the middle breaks this collapse. It bends the computation, letting the network approximate arbitrarily complex functions. Non-linearity is what makes depth worth having.

THE ACTIVATION FUNCTIONS YOU'LL MEET

- **ReLU** (Rectified Linear Unit): $g(x) = \max(0, x)$. Dead simple — keep positives, zero out negatives. A "bent" line, and that tiny bend is enough to unlock non-linearity. Fast and historically dominant.
- **GELU** (Gaussian Error Linear Unit): a smooth version of ReLU that doesn't have a hard corner at zero and lets slightly-negative values leak through a little. The default in GPT-style models because the smoothness helps learning.
- **SwiGLU**: a gated variant used in most recent open models (Llama, etc.). It uses an extra small gate to control how much of each feature passes, which empirically works better for the same compute.

You do not need the details to proceed. The one idea to keep: **a non-linear function sits between the two matrices, and without it the whole network would collapse to something trivial.**

5.9 Residual connections & LayerNorm

Two supporting mechanisms wrap around both attention and the FFN. They are not glamorous, but without them, deep transformers simply would not train.

Residual connections (the "add back")

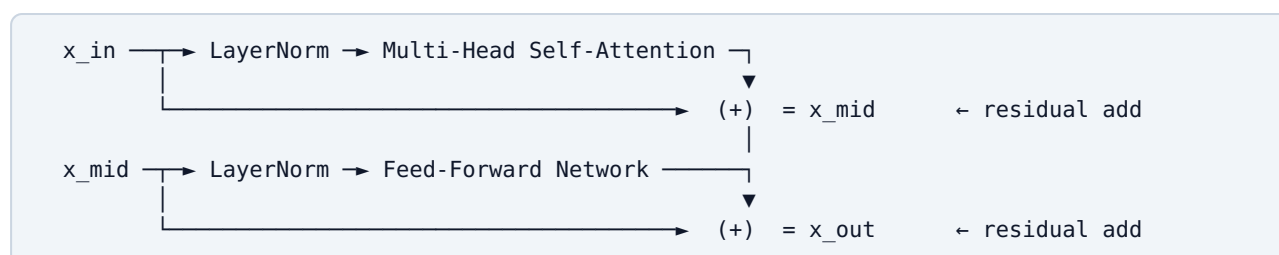
Instead of replacing a token's vector with the output of attention (or the FFN), the transformer **adds** that output back onto the original: x becomes $x + \text{Attention}(x)$. This is the "add back the input" arrow in the block diagram, and it is the residual stream from Part 4.3 in action. Two payoffs. First, each layer only has to learn a small *adjustment* to what came before, not rebuild everything — much easier. Second, it creates a clean highway for the learning signal to flow back down through dozens of layers during training (Part 8.4), preventing the signal from fading to nothing. Residual connections are the single trick that made very deep networks trainable.

Normalization (keeping numbers sane)

As vectors pass through many layers, their numbers can drift to be very large or very small, which destabilises training. **LayerNorm** (and its lighter cousin **RMSNorm**, now common) rescales each token's vector to a consistent, well-behaved range before it enters attention or the FFN — roughly, re-centering and re-sizing the numbers so no single feature blows up. Think of it as a thermostat that keeps the signal in the comfortable zone where learning works. Modern models place it *before* each sub-layer ("pre-norm"), which trains more stably than the original "post-norm" design.

5.10 The full block, assembled

We can now read the complete transformer block as a precise sequence, and it should feel earned:



The output x_{out} is the same shape as the input, so blocks stack cleanly. Attention lets tokens share; the FFN lets each token think; residuals and norm keep it stable and trainable. Stack this block N times (24, 32, 80, ...) and you have the body of an LLM. A final linear layer then maps each top-of-stack vector to V logits, and softmax turns those into the next-token probabilities from Part 1.

5.11 Encoder, decoder, encoder-decoder

Three arrangements exist, and the difference is just the masking and the goal:

Type	Attention	Used for	Examples
Decoder-only	Causal (past only)	Generating text	GPT, Claude, Llama, Gemini
Encoder-only	Bidirectional (both ways)	Understanding / embedding	BERT
Encoder-decoder	Both, joined by cross-attention	Translation, summarisation	T5, original transformer

Chat LLMs are **decoder-only**: one causal stack that both reads the prompt and writes the reply. This is the architecture you should picture whenever "LLM" is said unqualified.

KEY TAKEAWAYS

- A transformer is a stack of identical blocks; each has **attention** then a **feed-forward network**.
- Attention scores relevance with **dot products** of Queries and Keys, then blends Values.
- **Softmax** = exponentiate, then normalise: turns raw scores into positive weights summing to 1, amplifying the largest.
- The full attention equation scales scores by $\sqrt{d_k}$ before softmax for stability.
- **Multi-head** attention runs several attentions in parallel so each learns a different relationship.
- A **causal mask** stops a generating model from seeing future tokens.
- The **FFN** = expand, apply a **non-linearity** (ReLU/GELU/SwiGLU), compress — and the non-linearity is what stops the network collapsing to one linear layer.
- **Residual connections** ("add back") and **normalization** are what make deep stacks trainable.

Attention at Scale

The attention of Part 5 is correct but expensive. Making it fast enough for long documents and cheap enough to serve millions of users required a series of clever engineering fixes. You will see these names on every model spec sheet.

6.1 The n^2 problem

Attention compares every token with every other token. For a sequence of n tokens, that is $n \times n$ comparisons. Double the length and you *quadruple* the work; this is called **quadratic** cost, written $O(n^2)$. For 1,000 tokens that is a million comparisons; for 100,000 tokens it is ten billion. This single fact is why long context is hard and expensive, and why much of the field's engineering targets this one bottleneck.

6.2 The KV cache: don't redo work

Recall that generation is a loop — predict a token, append it, predict again. Naively, each new step would recompute the Keys and Values for the entire sequence so far, repeating enormous work. The **KV cache** fixes this: once a token's Key and Value vectors are computed, they never change, so we store them and reuse them. Each new step only computes K and V for the *one* new token and looks up the rest. This turns generation from hopelessly slow into practical — but the cache lives in precious GPU memory and grows with every token, which is why long conversations consume so much memory and why the next two tricks exist.

6.3 MQA and GQA: shrinking the cache

The KV cache stores separate Keys and Values for every attention head. That is a lot of memory. Two variants trade a little quality for a much smaller cache:

- **Multi-Query Attention (MQA)** — all heads *share* a single set of Keys and Values (only the Queries stay separate). The cache shrinks dramatically, but sharing can cost some quality.
- **Grouped-Query Attention (GQA)** — the sensible middle ground now used almost everywhere (Llama, Mistral, and others): heads are split into a few groups, and each group shares its K and V. You capture most of MQA's memory savings while keeping most of full attention's quality.

When a spec sheet lists "8 KV heads" alongside "32 attention heads," that is GQA: 32 Query heads sharing 8 sets of Keys/Values.

6.4 FlashAttention: the same math, done cleverly

FlashAttention does not change the attention formula at all — it computes the exact same result, but reorganises the arithmetic to match how GPU memory physically works. The naive method wastes enormous time shuttling the giant $n \times n$ score grid to and from slow memory. FlashAttention keeps the work in the GPU's tiny fast memory and never writes the full grid out, making attention several times faster and letting far longer sequences fit. It is a pure implementation win, and its adoption was near-universal almost overnight.

6.5 Sparse and sliding-window attention

Another angle: what if a token does not need to attend to *all* previous tokens? **Sliding-window attention** restricts each token to a fixed window of, say, the nearest 4,096 tokens, turning the quadratic cost into a linear one. Information still travels across long distances indirectly, because overlapping windows pass it along layer by layer, like a bucket brigade. Other **sparse** schemes let tokens attend to a chosen subset (e.g. every token can see a few "global" summary tokens). These trade a little reach for a lot of speed.

6.6 Long context in practice

Serving a 200,000-token context is a stack of the above: RoPE adjustments so positions still make sense (Part 4.5), GQA to keep the KV cache affordable, FlashAttention for speed, and sometimes windowed attention. Even so, two cautions carry into Part 13: long context is expensive (you pay per token, and the KV cache balloons), and a model given a huge context does not attend to all of it equally well — the middle is often neglected, a failure called "lost in the middle."

KEY TAKEAWAYS

- Attention is **quadratic** in sequence length — the root cause of long-context cost.
- The **KV cache** stores past Keys/Values so generation doesn't redo work; it eats GPU memory.
- **MQA/GQA** share Keys/Values across heads to shrink that cache; **GQA** is today's default.
- **FlashAttention** speeds up the exact same computation by respecting GPU memory layout.
- **Sliding-window / sparse** attention limits who each token sees, trading reach for speed.

Anatomy of an LLM: Reading the Spec Sheet

You asked specifically about "the different params in an LLM." Here it is, in full. After this part you will be able to read any model announcement — "70B parameters, 80 layers, 8k hidden, 64 heads, 128k context" — and know exactly what each number means and why it matters.

7.1 What "7B parameters" actually counts

A **parameter** is a single learned number — one weight inside one of the matrices we have been discussing. "7B parameters" means the model contains seven billion such numbers, all tuned during training. Parameters are the model's knowledge and skill, stored as numbers. More parameters generally means more capacity to learn, but also more memory, more compute, and more cost. Roughly, storing each parameter takes 2 bytes (at 16-bit precision), so a 7B model needs about 14 GB of memory just to hold its weights — before you run anything (see Part 11 for how quantization cuts this).

7.2 Where the parameters live

Those billions of parameters are not scattered randomly. They sit in a few well-defined places, and knowing the split demystifies the whole model:

Location	What it does	Share of params
Embedding matrix	Token ID → vector (Part 4)	Small-moderate
Attention ($W^{Q,K,V}$ + output)	Mixing info across tokens (Part 5)	~1/3 of each block
Feed-forward network	Per-token thinking & knowledge (Part 5.8)	~2/3 of each block
Output (un-embedding) layer	Final vector → logits (Part 1)	Small-moderate

The headline: the **feed-forward networks hold the majority of the parameters** — usually around two-thirds. This is consistent with the belief that FFNs store most of the model's factual knowledge, while attention handles the routing of information.

7.3 Depth, width, and heads — the three shape dials

- **Depth = number of layers (N).** How many transformer blocks are stacked. More layers allow more sequential steps of reasoning — more chances to refine the residual stream. Small models have ~12–32; large ones 80 or more.
- **Width = model dimension (d_{model}).** The length of each token's vector. Wider vectors hold more information per token. Typical values run 768 (small) to 8,192+ (large).
- **Heads = number of attention heads.** How many relationships attention can track in parallel per layer (Part 5.6). The width is divided among the heads.

These interact: total parameters grow roughly with $\text{depth} \times \text{width}^2$. Doubling the width quadruples the work of every matrix multiply, so width is the more expensive dial.

DEPTH VS. WIDTH, INTUITIVELY

Depth is *how many times* the model gets to think; width is *how much* it can hold in mind each time. Good architectures balance the two — a very deep but very narrow model, or a very wide but shallow one, both waste capacity. The best ratios were found largely by experiment.

7.4 Context window, vocabulary, and tied weights

- **Context window.** The maximum number of tokens the model can consider at once (prompt + output). This is not a parameter count — it is a limit on input length, governed by positional encoding and the memory/compute costs of Part 6. Ranges from a few thousand to over a million.
- **Vocabulary size (V).** How many distinct tokens exist (Part 3). It sets the size of both the input embedding and the output layer.
- **Weight tying.** Many models *share* the input embedding matrix and the output layer (they are transposes of each other), saving parameters and often improving quality, since the same "meaning ↔ token" mapping is used going in and coming out.

7.5 FLOPs: the compute, not the size

Parameters measure *size*; **FLOPs** (floating-point operations) measure *work*. A handy rule of thumb: generating one token with a dense model costs about $2 \times (\text{number of parameters})$ FLOPs. So a 7B model does roughly 14 billion operations per token generated. This is why bigger models are slower and pricier to run, and why the next idea — using only *part* of the model per token — is so attractive.

7.6 Mixture-of-Experts (MoE): total vs. active parameters

A **Mixture-of-Experts** model replaces each feed-forward network with *many* parallel FFNs (the "experts") plus a small **router** that, for each token, picks just a few experts to actually run. The model might contain, say, 8 experts per layer but use only 2 per token. This decouples two numbers you must not confuse:

- **Total parameters** — everything the model stores (all experts). Governs memory.
- **Active parameters** — the fraction actually used per token (the chosen experts). Governs speed and per-token cost.

So a model advertised as "8×7B" or "mixture of experts, 47B total / 13B active" is huge in knowledge (all experts hold information) yet fast to run (only some fire each token). MoE is how frontier labs push parameter counts into the hundreds of billions or trillions while keeping inference affordable. The trade-off: all those experts still must fit in memory, and routing adds complexity.

7.7 Reading a real spec sheet

DECODING A TYPICAL ANNOUNCEMENT

"Our model: 70B parameters, 80 layers, hidden size 8192, 64 attention heads (8 KV heads), 128k context, 128k vocabulary, SwiGLU, RoPE, GQA."

- **70B parameters** — 70 billion learned numbers; ~140 GB in 16-bit (Part 11 shrinks this).
- **80 layers** — 80 stacked transformer blocks; a deep model (Part 7.3).
- **hidden 8192** — each token vector has 8,192 numbers; a wide model.
- **64 heads, 8 KV heads** — 64 Query heads sharing 8 Key/Value sets → this is GQA (Part 6.3).
- **128k context** — up to 128,000 tokens in at once (Part 7.4).
- **128k vocab** — a large token vocabulary → efficient on text (Part 3.3).
- **SwiGLU** — the FFN's non-linearity (Part 5.8); **RoPE** — rotary positions (Part 4.5).

You now understand every term — that is the whole point of Parts 3 through 7.

KEY TAKEAWAYS

- A **parameter** is one learned weight; count $\approx$ capacity $\approx$ memory $\approx$ cost.
- Most parameters live in the **feed-forward networks**, which store most knowledge.
- **Depth** (layers) = how many thinking steps; **width** (d_{model}) = how much per step; **heads** = parallel relationships.
- **Context window** and **vocabulary size** are limits, not parameter counts; **weight tying** shares input and output tables.
- **FLOPs** $\approx 2 \times$ parameters per token — the cost of running the model.
- **MoE** splits **total** parameters (memory) from **active** parameters (speed) by routing each token to a few experts.

Pretraining: How a Model Learns

We have built the machine. Now: how do billions of random numbers become a model that knows things? This is **training**, and its engine is **backpropagation**, which we will build from absolute zero — loss, gradients, the chain rule, and the two passes.

8.1 The objective: predict the next token, a trillion times

Pretraining is conceptually simple. Take an enormous pile of text. Repeatedly: show the model some text with the next token hidden, let it predict, check whether it was right, and nudge its parameters to make the correct token slightly more likely next time. Do this trillions of times. The model starts as random noise producing gibberish; it ends as something that has, in effect, compressed the patterns of human text into its weights. No labels, no humans in the loop — the text is its own answer key, because the true next token is always right there. This is called **self-supervised** learning, and it is why we can train on raw internet text at massive scale.

8.2 Measuring wrongness: the loss and cross-entropy

To improve, the model needs a number that says *how wrong* each prediction was. That number is the **loss**. Lower is better; training is the process of making the loss small. For next-token prediction we use **cross-entropy loss**, and its idea is intuitive.

CROSS-ENTROPY, EXPLAINED

The model outputs a probability for the token that actually came next. Call that probability p . The loss for that one prediction is:

$$\text{loss} = -\log(p)$$

Read it through the behaviour of the logarithm:

- If the model was **confident and right** — it gave the true token $p = 0.99$ — then $-\log(0.99) \approx 0.01$. Almost no loss. Good.
- If it was **unsure** — $p = 0.5$ — then $-\log(0.5) \approx 0.69$. Moderate loss.
- If it was **confidently wrong** — it gave the true token only $p = 0.01$ — then $-\log(0.01) \approx 4.6$. Large loss.

So cross-entropy rewards putting high probability on the true token and punishes putting low probability on it, harshly when the model was confidently wrong. The total loss is the average of this over every predicted token in the batch.

YOU'LL HEAR "PERPLEXITY"

Perplexity is just e raised to the loss — a friendlier restatement. A perplexity of 10 means the model is, on average, as unsure as if it were choosing uniformly among 10 tokens. Lower perplexity = better model. It is the classic scoreboard for language models.

8.3 Gradient descent: rolling downhill

We have a loss that measures wrongness, and billions of parameters we can adjust. How do we choose adjustments that reduce the loss? Picture the loss as a vast landscape of hills and valleys, where your horizontal position is the setting of all parameters and your altitude is the loss. We want the lowest valley. We cannot see the whole landscape — but we can feel the **slope** under our feet and step downhill. Repeat, and we descend toward a valley. This is **gradient descent**.

THE GRADIENT AND THE UPDATE

The **gradient** is the slope of the loss with respect to a parameter: it answers "if I nudge this one weight up a little, does the loss go up or down, and how steeply?" Written $\partial \text{loss} / \partial w$ for a weight w (read "the change in loss per change in w "). Once we know the gradient for every parameter, we update each one by stepping in the *opposite* direction of its slope (downhill):

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial \text{loss}}{\partial w}$$

The small number η (eta) is the **learning rate** — the size of each downhill step. Too large and you overshoot the valley and bounce chaotically; too small and training crawls. Choosing it well is one of the dark arts of training.

So training reduces to: compute the gradient of the loss for every one of the billions of parameters, take a small downhill step, repeat millions of times. The only missing piece — and it is the crucial one — is *how* we compute those billions of gradients efficiently. That is backpropagation.

8.4 Backpropagation, in depth

The model is a long chain of operations: embed $\rightarrow$ attention $\rightarrow$ FFN $\rightarrow \dots \rightarrow$ logits $\rightarrow$ loss. A weight deep inside the network affects the loss only *indirectly*, through everything that comes after it. To find its gradient we need to trace that influence back through the whole chain. The tool for this is a rule from calculus you already use intuitively: the **chain rule**.

THE CHAIN RULE, IN ONE IDEA

If a affects b , and b affects c , then how much a affects c is the product of the two links:

$$\frac{\partial c}{\partial a} = \frac{\partial c}{\partial b} \times \frac{\partial b}{\partial a}$$

Intuition: if turning knob a moves b twice as fast, and moving b moves c three times as fast, then a moves c six times as fast. Effects **multiply** along a chain. Backpropagation is nothing more than applying this rule, link by link, from the loss all the way back to every weight.

The two passes

Every training step is a forward pass followed by a backward pass over the same example:

1. **Forward pass.** Push the input through the network normally, layer by layer, to produce the prediction and compute the loss. Along the way, remember the intermediate values at each step — you will need them in a moment.

2. **Backward pass.** Start at the loss with the trivial gradient "how does the loss change with the loss? By 1." Then walk *backward* through the network. At each operation, use the chain rule to convert "how the loss depends on my output" into "how the loss depends on my inputs and my weights." Hand the input-gradient to the previous layer and keep the weight-gradient for the update. Repeat until every layer has been visited.

The name says it all: gradients **propagate backward** from the loss to the parameters. Because each layer only does a small local calculation and passes the result along, the whole thing is efficient — one backward sweep computes gradients for *all* billions of parameters at once. And remember the residual connections from Part 5.9? They give this backward signal a clean highway, so it does not fade to nothing across dozens of layers — that is precisely why deep networks became trainable.

```
FORWARD  >>>  embed → attn → ffn → ... → logits → LOSS
                                     |
BACKWARD <<<  ∂L/∂w ← ∂L/∂w ← ∂L/∂w ← ... ← (start: ∂L/∂L = 1)
               (each layer: chain-rule its output-gradient into input- and weight-gradients)
```

ONE TRAINING STEP, END TO END

(1) Show the model "The capital of France is ____." (2) **Forward:** it predicts, giving the true token "Paris" a probability of only 0.2. (3) Loss = $-\log(0.2) \approx 1.6$ — fairly wrong. (4) **Backward:** backprop computes, for every weight, whether nudging it up or down would have raised "Paris"'s probability. (5) **Update:** step every weight a tiny amount in the helpful direction. Next time this example appears, "Paris" scores a hair higher. Multiply by trillions of examples and the model learns the capital of France — and grammar, and code, and reasoning — all through this same loop.

THE OPTIMIZER: ADAM

Plain gradient descent works, but modern training uses a smarter update rule called **Adam** (or AdamW). It keeps a short memory of recent gradients per parameter and adapts each parameter's step size individually — giving momentum through flat regions and caution on steep ones. You will see "AdamW" in every training recipe; it is gradient descent with these sensible refinements.

8.5 The data pipeline

A model is what it eats. Frontier pretraining runs on *trillions* of tokens drawn from web pages, books, code, and more — but raw web text is filthy, so most of the effort is cleaning:

- **Filtering** low-quality, spam, and toxic content.
- **Deduplication** — removing repeated documents, which otherwise waste compute and cause memorisation.
- **Mixing** — deliberately balancing sources (more code improves reasoning; more books improve long-form coherence). The recipe is a closely guarded ingredient.

The maxim "garbage in, garbage out" is doubly true at scale: data quality now rivals model size as a driver of capability.

8.6 The compute: why this needs a data centre

Training a frontier model means doing the forward/backward loop over trillions of tokens with billions of parameters — far more than any single chip can hold or finish in a lifetime. So the work is split across thousands of GPUs using three kinds of parallelism, often combined:

- **Data parallelism** — each GPU holds a full copy of the model and processes a different slice of data; gradients are averaged across GPUs.
- **Tensor parallelism** — a single matrix multiply is split across several GPUs when one layer is too big for one chip.
- **Pipeline parallelism** — different GPUs hold different layers, passing activations along like an assembly line.

This is why training a large model can cost millions of dollars and weeks of wall-clock time on specialised clusters — and why only a handful of organisations train the very largest models from scratch.

8.7 Scaling laws: bigger, predictably better

A remarkable discovery: model quality improves in a smooth, predictable way as you increase three things together — **parameters**, **data**, and **compute**. Plotted on the right axes, the loss falls along a straight line over many orders of magnitude. This means you can *predict* a big model's performance from small experiments before spending the money — which is exactly how labs decide what to build.

The influential **Chinchilla** result (2022) sharpened this: for a fixed compute budget, most early large models were *under-trained* — too many parameters, too little data. The compute-optimal recipe is to scale parameters and training tokens *together*, roughly in balance (a rule of thumb near 20 tokens per parameter). This reshaped the field toward smaller models trained on far more data, which is why capable 7B-8B models exist today.

8.8 Emergence: the controversial part

Some abilities appear to switch on suddenly at scale — a model too small cannot do multi-step arithmetic or certain reasoning at all, then past some size it can. These are called **emergent abilities**. Whether they are truly abrupt or just look that way because of how we measure them is actively debated. Either way, the practical lesson stands: capabilities are not fully predictable from small models, and scale keeps surprising us.

KEY TAKEAWAYS

- Pretraining is **self-supervised**: predict the next token trillions of times; the text is its own answer key.
- The **loss** (cross-entropy, $-\log p$) measures wrongness; training minimises it.
- **Gradient descent** steps every parameter downhill along its slope; the **learning rate** sets the step size.
- **Backpropagation** uses the **chain rule** in a backward pass to get all gradients in one efficient sweep.
- Residual connections keep the backward signal alive through deep stacks.
- **Data quality**, **distributed compute**, and **scaling laws** (Chinchilla balance) govern how good the result is.

From Base Model to Assistant

Pretraining produces a brilliant autocomplete that is not yet an assistant. Post-training — a much smaller, more targeted phase — turns raw capability into something helpful, honest, and safe to talk to.

9.1 Why a base model isn't an assistant

Ask a base model "What is the capital of France?" and it might reply with a *list of more trivia questions* — because on the internet, a question is often followed by more questions. The base model is not being unhelpful; it is doing its job (predicting plausible text) perfectly. It simply has no notion that *you want it to answer*. Post-training installs that notion.

9.2 Supervised fine-tuning (SFT)

The first step is **supervised fine-tuning**, also called instruction tuning. We collect thousands of high-quality examples of the desired behaviour — an instruction paired with an ideal response, often written by skilled humans — and continue training (same next-token objective, same backprop) on *these*. The model quickly learns the *format* of being helpful: when it sees an instruction, it should produce a direct, well-structured answer. SFT is powerful and cheap relative to pretraining, but it can only imitate the examples it is shown.

9.3 Learning from preferences: RLHF

SFT teaches the model to imitate good answers; it does not teach it to tell a *great* answer from a merely *okay* one. For that we use human *preferences*. **RLHF** — Reinforcement Learning from Human Feedback — works in two stages:

1. **Train a reward model.** Show humans two model responses to the same prompt and ask which they prefer. Collect many such comparisons, then train a separate model — the **reward model** — to predict human preference, outputting a score for any response.
2. **Optimise against it.** Now let the LLM generate responses and adjust its parameters to earn higher scores from the reward model, using a reinforcement-learning algorithm (classically **PPO**). A guardrail keeps it from drifting too far from the sensible SFT model while it chases reward.

The result is a model tuned not just to imitate but to produce answers humans actively prefer — clearer, more helpful, more appropriately hedged. RLHF is the ingredient that made ChatGPT feel qualitatively different from raw GPT-3.

9.4 A simpler path: DPO

RLHF's two-model, reinforcement-learning machinery is finicky. **Direct Preference Optimization (DPO)** achieves a similar goal with a single, more stable training step: it uses the same "preferred vs. rejected" pairs but skips the separate reward model and the RL loop, directly nudging the model to make preferred answers more likely and rejected ones less likely. Because it is simpler and cheaper, DPO and its relatives are now widely used, especially in open models.

9.5 Constitutional AI and RLAIIF

Human feedback is slow and expensive to collect. **Constitutional AI** (developed at Anthropic) reduces the human load by giving the model a written set of principles — a "constitution" — and having the *model itself* critique and revise its answers against those principles. AI-generated preference labels then stand in for many human ones (**RLAIF**: RL from AI Feedback). This scales alignment and makes the guiding values explicit and inspectable rather than buried in thousands of individual human judgments.

9.6 What alignment does — and doesn't — fix

Post-training makes a model helpful, better at following instructions, more honest about uncertainty, and more likely to refuse genuinely harmful requests. What it does **not** do is add new knowledge the base model never learned, or eliminate hallucination, or guarantee the model can never be tricked. It shapes *behaviour and priorities*, working with the capabilities pretraining already created. Keep this boundary in mind for Part 13: many limitations are properties of the base model that alignment can soften but not remove.

KEY TAKEAWAYS

- A **base model** autocompletes; post-training makes it an **assistant**.
- **SFT** teaches the format of helpfulness by imitating ideal instruction-response pairs.
- **RLHF** trains a **reward model** from human preferences, then optimises the LLM to score well.
- **DPO** reaches a similar result more simply, without a separate reward model or RL loop.
- **Constitutional AI / RLAIF** scale alignment using explicit principles and AI-generated feedback.
- Alignment shapes **behaviour**, not knowledge — it can't cure hallucination or add facts.

Inference & Decoding

Training is done; the weights are frozen. **Inference** is using the model to generate text. The model gives a probability distribution over the next token — but which token do we actually pick? That choice, called **decoding**, quietly controls whether output feels robotic, balanced, or unhinged.

10.1 The autoregressive loop, revisited

Generation is the loop from Part 1, now with everything under it understood: feed the prompt through the transformer, get logits for the next token, apply softmax for probabilities, *choose* one token, append it, and repeat — reusing the KV cache (Part 6.2) so each step is cheap. The loop stops when the model emits the special end-of-sequence token or hits a length limit. All the interesting choices live in that one word, "*choose*."

10.2 Greedy and beam search

- **Greedy decoding.** Always take the single highest-probability token. Fast and deterministic, but short-sighted — grabbing the best token *now* can lead into a worse sentence later, and it tends to be repetitive and bland.
- **Beam search.** Keep several candidate continuations (say 4 "beams") alive at once and expand the most promising, ultimately choosing the highest-probability *whole sequence*. Better for tasks with one correct answer (translation), but for open-ended writing it produces oddly generic, repetitive text — and it's slower. Rarely used for chat.

10.3 Sampling and temperature

Instead of always taking the top token, we can **sample** — roll a weighted die according to the probabilities, so a token with 0.3 probability is chosen 30% of the time. This injects variety, the lifeblood of natural, creative text. How *much* variety is set by **temperature**, the softmax knob from Part 5.4:

Temperature	Effect on the distribution	Feel of the output
Low ($\rightarrow 0$)	Sharpens toward the top token	Focused, deterministic, repetitive
Around 0.7–1.0	Keeps the natural distribution	Balanced, natural (default range)
High (> 1.2)	Flattens toward uniform	Creative, surprising, error-prone

Low temperature for factual answers and code; higher for brainstorming and fiction. Temperature 0 is effectively greedy decoding.

10.4 Top-k, top-p, and min-p

Pure sampling has a danger: even a truly terrible token has a tiny probability, and once in a while the die lands on it, derailing the text. So we *truncate* the distribution first, keeping only sensible candidates:

- **Top-k.** Keep only the k most likely tokens (e.g. top 40), discard the rest, renormalise, then sample. Simple, but a fixed k is clumsy: sometimes 40 tokens are all reasonable, sometimes only 2 are.
- **Top-p (nucleus).** Keep the smallest set of top tokens whose probabilities *add up* to p (e.g. 0.9), then sample from those. The candidate pool grows and shrinks automatically with the model's confidence — the most popular method.
- **Min-p.** Keep tokens whose probability is at least some fraction of the top token's probability. A newer, robust alternative that adapts naturally to how peaked the distribution is.

In practice these combine: apply top-p (or top-k), then temperature, then sample. These are the "knobs" exposed in most APIs.

10.5 Controlling repetition

Models sometimes fall into loops ("the the the") or restate themselves. **Repetition penalties** and **frequency/presence penalties** lower the probability of tokens that have already appeared, gently steering the model toward fresh words. Useful in moderation; too much makes text feel forced and disjointed.

10.6 Prefill vs. decode: two very different phases

Serving a request has two phases with opposite cost profiles, which explains a lot about pricing and latency:

- **Prefill.** Reading your prompt. All prompt tokens are processed *in parallel* in one efficient pass to populate the KV cache. Fast per token, but it is why a huge prompt has an upfront cost and why "time to first token" grows with prompt length.
- **Decode.** Writing the response, one token at a time, each step depending on the last. This is inherently sequential and cannot be parallelised across the tokens being generated, so it dominates the time for long outputs.

This is why input tokens and output tokens are often priced differently, and why generating 1,000 tokens takes far longer than reading 1,000.

10.7 Speeding up decode: batching and speculation

- **Batching.** A GPU is happiest doing lots of work at once, so servers process many users' requests together. This is the main lever for high throughput and low cost per token.
- **Speculative decoding.** A clever trick: a small, fast "draft" model guesses several tokens ahead, and the big model *checks* all of them in a single parallel pass, accepting the ones it agrees with. Because verifying is cheaper than generating, the big model produces several tokens for roughly the price of one — a sizeable speed-up with *identical* output quality.

KEY TAKEAWAYS

- **Decoding** is how we pick a token from the model's probability distribution each step.
- **Greedy/beam** are deterministic; **sampling** adds the variety natural text needs.
- **Temperature** sets randomness; **top-p** (nucleus) is the common way to cut off nonsense tokens.
- **Prefill** (reading, parallel) and **decode** (writing, sequential) have opposite cost profiles.
- **Batching** and **speculative decoding** make serving fast and cheap without hurting quality.

Making Models Smaller & Faster

A 70B model in full precision needs ~140 GB of memory — more than most GPUs have — and is slow and costly to run. A family of compression techniques makes big models fit on smaller hardware and run faster, mostly without ruining quality.

11.1 Quantization: fewer bits per number

Every parameter is stored as a number with some **precision** — how many bits represent it. Training typically uses 16-bit numbers. **Quantization** stores them with fewer bits — 8-bit or even 4-bit — roughly halving or quartering the memory. The surprise is how little quality is lost: a well-quantized 4-bit model is often nearly as good as the original while using a quarter of the memory, because neural networks are robust to small rounding of their weights.

THE MEMORY MATHS

Memory for weights $\approx$ (parameters) $\times$ (bytes per parameter).

- 16-bit (2 bytes): 7B model $\rightarrow$ ~14 GB
- 8-bit (1 byte): 7B model $\rightarrow$ ~7 GB
- 4-bit (0.5 byte): 7B model $\rightarrow$ ~3.5 GB — now it fits on a consumer GPU

Methods like **GPTQ** and **AWQ** quantize carefully — protecting the most sensitive weights — to keep quality high at 4-bit. This is why you can run capable models on a laptop.

11.2 Distillation: a small student learns from a big teacher

Knowledge distillation trains a small "student" model to imitate a large "teacher" model's outputs (its full probability distributions, which carry more information than a bare answer). The student ends up far better than if it had trained on raw data alone, capturing much of the teacher's ability at a fraction of the size. Many small, capable models you can run locally are distilled from larger ones.

11.3 Pruning: cutting dead weight

Pruning removes parameters that contribute little — setting near-useless weights to zero, or removing whole neurons or heads — then briefly retrains to recover. Networks are often over-provisioned, so a surprising fraction can be cut with minor quality loss. Pruning is finickier to get real speed-ups from than quantization, so it is used more selectively.

11.4 LoRA and QLoRA: cheap fine-tuning

The techniques above shrink a model for *running*. **LoRA** (Low-Rank Adaptation) instead makes *fine-tuning* cheap. Fully fine-tuning a large model means updating all its billions of parameters — expensive in memory and compute. LoRA freezes the original weights and inserts tiny "adapter" matrices, training *only* those — often less than 1% of the parameters. You get most of the benefit of full fine-tuning for a tiny fraction of the cost, and you can keep many small adapters for different tasks over one shared base model.

QLoRA combines this with quantization: keep the frozen base model in 4-bit to save memory, and train small LoRA adapters on top. Together they let people fine-tune large models on a single GPU — a major reason open-model customisation exploded.

KEY TAKEAWAYS

- **Quantization** stores weights in fewer bits (8- or 4-bit), cutting memory with little quality loss.
- **Distillation** trains a small student to mimic a big teacher, transferring much of its ability.
- **Pruning** removes low-value parameters; effective but harder to turn into real speed-ups.
- **LoRA** fine-tunes cheaply by training tiny adapters; **QLoRA** adds quantization to fit on one GPU.

Using LLMs Well

You now understand the machine. This part is about driving it: how to get good results from a fixed, trained model without changing a single weight — entirely through what you put in the context window.

12.1 In-context learning: teaching by example

One of the most surprising properties of large models: you can teach them a task *at inference time*, just by showing examples in the prompt — no training, no weight updates. Give a model three examples of turning a sentence into JSON, and it will do the fourth. This is **in-context learning**, and it emerged from scale (Part 2.8). The vocabulary:

- **Zero-shot** — just the instruction, no examples.
- **Few-shot** — the instruction plus a handful of worked examples. Often dramatically better for a specific format or style.

The model isn't learning in the training sense; it is recognising the pattern in your prompt and continuing it. But from the outside it behaves like rapid, throwaway learning — a genuinely useful tool.

12.2 Chain-of-thought: giving the model room to think

Ask a model a hard reasoning question and demand only the final answer, and it often fumbles — because producing the answer requires several steps, and each generated token has a fixed, small amount of computation behind it. Ask it to "think step by step" and it will lay out intermediate reasoning before answering, and accuracy jumps. This is **chain-of-thought** prompting.

WHY IT WORKS

The model can only do so much computation per token. By writing out its reasoning, it effectively uses *more tokens* — more total computation — and each step conditions the next, so it can build toward an answer it could not reach in one leap. The written trace is scratch paper. Part 14 shows how the newest "reasoning models" bake this habit in and take it much further.

12.3 System prompts: setting the stage

Chat models accept a special **system prompt** — instructions, delivered via the role tokens from Part 3.4, that set persona, rules, and context before the conversation starts ("You are a terse technical assistant; prefer code to prose"). Because it sits at the front of the context and is framed as authoritative, it strongly shapes everything after. Clear, specific system prompts are one of the highest-leverage tools for controlling behaviour.

12.4 Tools and structured output

An LLM is a text predictor; it cannot truly do arithmetic, fetch a live web page, or run code by itself. **Tool use** (a.k.a. function calling) fixes this: the model is told which tools exist, and when it needs one it emits a structured request ("call `calculator(12.4 * 88)`"); the

surrounding software runs the tool and feeds the result back into the context for the model to use. This is how assistants get calculators, web search, databases, and code execution — and it is the foundation of **agents** (Part 14.3).

Relatedly, models can be constrained to emit **structured output** — strictly valid JSON, for example — so their responses can be consumed reliably by other programs rather than parsed from free text.

12.5 The bridge to RAG

Everything in this part manipulates the context window — and the most important context-manipulation technique of all is the subject of this guide's companion volume. **Retrieval-Augmented Generation (RAG)** fetches relevant documents at query time and inserts them into the prompt, so the model can answer from *your* data, cite sources, and stay current — directly addressing the knowledge-cutoff and hallucination limits we turn to next.

HOW THE TWO GUIDES CONNECT

This guide explains the **generator** — the LLM that reads a prompt and writes an answer. The RAG guide explains how to **build the prompt**: load and chunk documents, embed them (Part 4's idea, applied to whole passages), store them in a vector database, retrieve the right ones for a question, and hand them to the LLM. An LLM is the engine; RAG is the fuel line that feeds it the right facts at the right moment. Read them together and you can build a complete, grounded question-answering system end to end.

KEY TAKEAWAYS

- **In-context learning** teaches a task via prompt examples — zero-shot or few-shot — with no training.
- **Chain-of-thought** ("think step by step") buys more computation and boosts reasoning.
- The **system prompt** sets persona and rules and strongly steers behaviour.
- **Tool use** lets the text-only model call calculators, search, and code; the base of **agents**.
- **RAG** (the companion guide) feeds retrieved, cited facts into the context to fight hallucination and staleness.

Capabilities, Limits & Failure Modes

A clear-eyed user knows not just what LLMs can do but *why* they fail. Most limitations follow directly from how the model works — which is why the earlier parts were worth the effort.

13.1 Why hallucination is structural

A **hallucination** is a confident, fluent statement that is simply false. It is tempting to see this as a bug, but it follows from the core design: the model is trained to produce *plausible* text, not *true* text. It has no separate database of facts to check against — knowledge is smeared across billions of weights as statistical tendencies. When it doesn't "know" something, it does not fall silent; it generates the most plausible-sounding continuation, which can be a fluent fabrication. Post-training reduces this (teaching the model to hedge and admit uncertainty) and RAG reduces it further (supplying real sources), but nothing removes it entirely, because plausibility and truth are not the same target.

13.2 Knowledge cutoff

A model's knowledge is frozen at the moment its training data was collected — its **knowledge cutoff**. It cannot know anything after that date: yesterday's news, a library released last month, the current price of anything. Worse, it may not *know* that it doesn't know, and answer with stale confidence. Tool use and RAG are the standard fixes, connecting the frozen model to live information.

13.3 Lost in the middle

Even within its context window, a model does not use all positions equally. Given a long document, models reliably use information at the *beginning* and *end* but often overlook facts buried in the *middle* — the "lost in the middle" effect. So a bigger context window is not a guarantee that everything in it will be used; placement matters. Put the most important material where the model looks hardest.

13.4 Arithmetic and multi-step reasoning

Models are unreliable at exact arithmetic and long logical chains, for two reasons we can now name. First, tokenization hides digit structure (Part 3.5), so numbers aren't cleanly represented. Second, each token carries a fixed, limited amount of computation (Part 12.2), so problems needing many careful steps exceed what one forward pass can do. Chain-of-thought helps by spreading the work over more tokens, and tool use (a real calculator) fixes arithmetic outright — which is exactly why serious systems give models tools rather than trusting their mental math.

13.5 Bias, safety, and jailbreaks

- **Bias.** A model trained on human text absorbs human biases present in that text. Post-training mitigates the worst of it, but no model is neutral; outputs reflect their data.

- **Jailbreaks.** Cleverly crafted prompts can sometimes coax a model past its safety training (e.g. by role-play or obfuscation). Defending against this is an ongoing arms race, not a solved problem.
- **Prompt injection.** When a model reads untrusted content (a web page, a document, a tool result), hidden instructions in that content can hijack its behaviour. This is a serious, unsolved security concern for any system that lets a model act on external text — and a direct risk for tool-using agents and RAG systems.
- **Sycophancy.** Because RLHF optimises for what humans *rate highly*, models can drift toward telling people what they want to hear rather than what is accurate. Worth watching for whenever you rely on a model's agreement.

13.6 Benchmarks and why they mislead

Model quality is measured on **benchmarks** — standard test sets like MMLU (broad knowledge), GSM8K (grade-school math), HumanEval (coding), and many more. They are useful but leaky:

- **Contamination.** If a benchmark's questions leaked into training data, high scores may reflect memorisation, not ability.
- **Overfitting to the test.** When a benchmark becomes a target, labs optimise for it, and the score stops tracking real-world usefulness (Goodhart's law).
- **Narrowness.** A score on one test says little about the messy, open-ended tasks people actually use models for.

Treat leaderboard numbers as rough signals, not verdicts. The best evaluation is still trying a model on *your* real task — a lesson the RAG guide's evaluation section echoes in depth.

KEY TAKEAWAYS

- **Hallucination** is structural: models optimise for plausible, not true, text.
- Knowledge is frozen at the **cutoff**; tools and RAG connect to live facts.
- **Lost in the middle:** long context isn't used evenly — placement matters.
- Exact **arithmetic and long reasoning** are weak; use chain-of-thought and real tools.
- Watch for **bias, jailbreaks, prompt injection, and sycophancy** — none are fully solved.
- **Benchmarks** are leaky signals; test on your own task.

The Frontier

The fundamentals in this guide are stable, but the field moves fast. These are the directions defining the current generation of models. Specific model names date quickly; the *ideas* below are what to understand.

14.1 Reasoning models and test-time compute

The newest and most consequential shift: models that are trained to **think for longer before answering**. Where a standard model spends a fixed, small amount of computation per token, **reasoning models** generate a long internal chain of thought — sometimes thousands of hidden tokens of working — before producing a final answer. This is **test-time compute**: spending more computation at the moment of answering (rather than only at training time) to solve harder problems. The trade-off is latency and cost for accuracy, and it has pushed performance on math, coding, and logic far beyond what one-shot generation could reach. It essentially bakes Part 12.2's chain-of-thought into the model and scales it up, reinforced by training that rewards correct reasoning.

14.2 Multimodality

Text is only one kind of token. **Multimodal models** extend the same transformer machinery to images, audio, and video by converting those into tokens too — an image becomes a grid of patch "tokens," audio becomes chunks — and feeding them into the shared attention stream alongside text. One model can then look at a photo and answer questions about it, transcribe speech, or read a chart. The architecture barely changes; the tokenizer and the training data do.

14.3 Agents

An **agent** is an LLM placed in a loop with tools (Part 12.4) and a goal, where the model plans, takes an action via a tool, observes the result, and decides the next action — repeating until the task is done. Instead of answering a question, it *accomplishes a task*: research across many web pages, edit files across a codebase, book a series of steps. Agents are powerful but inherit and amplify every limitation in Part 13 — a small per-step error rate compounds over many steps, and prompt injection becomes a live security risk when the agent reads untrusted content. Reliability at long horizons is the central open challenge.

14.4 Frontier-scale Mixture-of-Experts

The largest models increasingly use **Mixture-of-Experts** (Part 7.6) to reach enormous total parameter counts — hundreds of billions to trillions — while keeping the *active* parameters per token, and thus the cost, manageable. This is a major reason capability has kept climbing without inference cost climbing at the same rate. Expect the gap between "total" and "active" parameters to keep widening.

14.5 Open weights vs. closed

Two ecosystems coexist. **Closed** models (accessed only through an API) tend to lead at the frontier and are easier to serve and govern centrally. **Open-weight** models (whose parameters are downloadable) can be run privately, fine-tuned freely (Part 11.4), inspected, and used offline — trading some peak capability for control, privacy, and cost. The gap between the best open and best closed models has been narrowing. Which to choose depends on your needs: data sensitivity, customisation, budget, and how close to the absolute frontier you must be.

KEY TAKEAWAYS

- **Reasoning models** spend more compute at answer time (long internal chains) for harder problems.
- **Multimodality** turns images/audio/video into tokens and reuses the same transformer.
- **Agents** loop an LLM with tools to accomplish tasks; errors and prompt injection compound over steps.
- **Frontier MoE** grows total parameters while holding active cost down.
- **Open-weight vs. closed** is a trade-off of control and privacy against peak capability.

A Beginner's Checklist & Glossary

A one-page map of how every piece fits, followed by a plain-language dictionary of every term in this guide. Come back here whenever a word feels slippery.

15.1 The whole pipeline, in one breath

```
— BUILD THE MODEL (once, at huge cost) —  
raw text → TOKENIZE (Part 3) → EMBED + POSITION (Part 4)  
    → TRANSFORMER STACK: [ attention + feed-forward ] × N (Part 5)  
    → trained by NEXT-TOKEN PREDICTION via BACKPROP (Part 8)  
    → then POST-TRAINED into an assistant: SFT + RLHF/DPO (Part 9)  
  
— USE THE MODEL (every request) —  
your prompt → (optionally + RAG documents, Part 12.5) → model forward pass  
    → logits → SOFTMAX → DECODE a token (temperature, top-p, Part 10)  
    → append, repeat → the answer
```

15.2 If you learn in this order, nothing is confusing

1. Next-token prediction & probability (Part 1) — the one idea everything rests on.
2. Tokenization & embeddings (Parts 3–4) — how text becomes vectors.
3. Attention & softmax (Part 5.2–5.7) — how tokens share context.
4. Feed-forward, residuals, norm (Part 5.8–5.10) — the rest of the block.
5. Loss, gradient descent, backprop (Part 8) — how it learns.
6. SFT & RLHF (Part 9) — how it becomes an assistant.
7. Decoding (Part 10) & limits (Part 13) — how to use it wisely.

15.3 Glossary

Term	Plain meaning
Token	A chunk of text (a word or word-piece); the unit a model reads and writes.
Tokenization	Splitting text into tokens; BPE learns the pieces by merging frequent pairs.
Embedding	A learned vector of numbers representing a token's meaning.
Vector	A list of numbers; a point in space. Similar meanings sit close together.
Logits	Raw, unnormalised scores the model outputs, one per vocabulary token.
Softmax	Turns logits into probabilities (all positive, summing to 1) by exponentiating and normalising.
Autoregressive	Generating one token at a time, each feeding back as input for the next.
Transformer	The architecture behind all LLMs: a stack of attention + feed-forward blocks.
Attention	Mechanism letting each token gather info from relevant other tokens.
Q, K, V	Query (what I seek), Key (what I offer), Value (info I hand over) — the vectors attention uses.
Multi-head	Running several attentions in parallel so each learns a different relationship.
Causal mask	Blocks a generating model from attending to future tokens.
Feed-forward (FFN/MLP)	Per-token processing: expand, apply a non-linearity, compress. Holds most knowledge.
Non-linearity	A function (ReLU/GELU/SwiGLU) that stops stacked layers collapsing into one.
Residual connection	Adding a layer's input back to its output; makes deep nets trainable.
LayerNorm / RMSNorm	Rescales vectors to a stable range so training doesn't blow up.
Parameter / weight	One learned number inside the model; billions of them make up its knowledge.
d_{model} (width)	Length of each token vector — how much info per token.
Layers (depth)	How many transformer blocks are stacked — how many thinking steps.
Context window	Max tokens the model can consider at once (prompt + output).
RoPE	Rotary position encoding: rotates Q/K by position, encoding relative distance.
KV cache	Stored past Keys/Values so generation doesn't redo work; uses GPU memory.
GQA / MQA	Sharing Keys/Values across heads to shrink the KV cache.
MoE	Many expert FFNs with a router; only a few run per token (total vs. active params).

Loss	A number measuring how wrong a prediction is; training minimises it.
Cross-entropy	The loss for prediction: $-\log(\text{probability of the true token})$.
Gradient	The slope of the loss w.r.t. a weight: which way, and how much, to nudge it.
Gradient descent	Repeatedly stepping every weight downhill to reduce the loss.
Learning rate	The size of each downhill step.
Backpropagation	Using the chain rule in a backward pass to get all gradients at once.
Pretraining	The big, self-supervised next-token-prediction phase that builds capability.
SFT	Supervised fine-tuning: teaching helpful behaviour from ideal examples.
RLHF	Reinforcement Learning from Human Feedback: tuning toward human-preferred answers.
DPO	A simpler alternative to RLHF using preference pairs directly.
Temperature	Decoding knob for randomness: low = focused, high = creative.
Top-p (nucleus)	Sampling only from the smallest set of tokens covering probability p.
Quantization	Storing weights in fewer bits (8/4-bit) to save memory.
LoRA / QLoRA	Cheap fine-tuning via small adapters (QLoRA adds quantization).
Hallucination	A confident but false output; structural, because models optimise for plausibility.
In-context learning	Teaching a task via prompt examples, with no weight changes.
Chain-of-thought	Prompting the model to reason step by step, improving accuracy.
Agent	An LLM in a plan-act-observe loop with tools, pursuing a goal.
RAG	Retrieval-Augmented Generation: feeding retrieved documents into the prompt (companion guide).

WHERE TO GO NEXT

You now hold the full mental model of a large language model — from a single token to the frontier. The natural next step is the companion **RAG Study Guide**, which shows how to feed an LLM the right facts at the right time to build grounded, cited, production-ready systems. The engine is yours; go build something that runs on it.